

GameFlowManager: GameState

i Для управления циклом всей игры необходим некий глобальный менеджер, который будет содержать “кнопки” для управления большинством других менеджеров. Тот, кто будет знать текущее состояние игры и менять их, сохраняя логику в норме.

GameFlowManager знает о текущем состоянии игры и сам же контролирует эти состояния. Разберём цикл всех состояний:

1. LoadStage() - именно этот метод срабатывает при нажатии старт в главном меню. Он подгружает случайный уровень, сбрасывает кошелек, устанавливает текущий этаж в менеджере сложности, показывает HUD, сообщает спавнеру, что нужно инициализироваться. Появляется кнопка “Нажмите чтобы начать”.
2. StartPlaying() - запускается при нажатии на эту кнопку. Начинается сама игра.
3. EscMenu() - запускается при нажатии кнопки на HUD, либо по нажатию на ESC.
4. LeaveGame() - срабатывает по нажатию кнопки выхода и следом вызывает MenuGame()
5. MenuGame() - выкидывает нас на главное меню.
6. ReturnPlaying() - ещё одна нажатие ESC или нажатие на кнопку return - продолжает игру.
7. DefeatGame() - в случае если здоровье базы опустилось до 0, то мы проиграли. У нас открывает DeathScreen, где мы можем - попробовать снова или покинуть игру.
8. RestartGame() - кнопка на экране проигрыша - перезапускает игру заного.
9. OnStageCompleted() - победа на этаже - запускает окно выбора артефакта.
10. ContinueAfterArtifact() - перейти в следующий этаж.

```
1 using System;
2 using Unity.Android.Gradle;
3 using Unity.VisualScripting;
4 using UnityEngine;
5 using UnityEngine.InputSystem;
6 using UnityEngine.SceneManagement;
7
8 public enum GameState
9 {
10     Menu,
11     Loading,
12     Playing,
13     ArtifactChoice,
14     GameOver,
15     Prestart,
16     EscMenu
```

```

17 }
18
19
20 public class GameFlowManager : Manager<GameFlowManager>
21 {
22     public GameState State { get; private set; } = GameState.Menu;
23
24     private bool restartLocked = false;
25
26     private void Start()
27     {
28         BaseManager.Instance.OnBaseDestroyed += GameOver;
29     }
30
31     private void OnDisable()
32     {
33         BaseManager.Instance.OnBaseDestroyed -= GameOver;
34     }
35
36     public void StartRun()
37     {
38         GameManager.Instance.AddFloor();
39         LoadStage();
40     }
41
42     public void OnStageCompleted()
43     {
44         State = GameState.ArtifactChoice;
45         SpeedGameManager.Instance.Pause();
46         HudManager.Instance.ShowStageCompletedCanvas();
47         AudioManager.Instance.PlaySFXType(SoundDefaultEnum.WinFloor);
48     }
49
50     public void ContinueAfterArtifact()
51     {
52         GameManager.Instance.AddFloor();
53         LoadStage();
54     }
55
56     private void LoadStage()
57     {
58         if (State == GameState.Loading) return;
59         State = GameState.Loading;
60
61         AudioManager.Instance.StopMusic();
62
63         SceneLoader.Instance.LoadScene(
64             LevelManager.Instance.GetRandomLevel(),
65             () =>
66             {
67                 restartLocked = false;
68
69                 DifficultyManager.Instance.SetFloor(GameManager.Instance.Floor);
70                 WalletManager.Instance.ResetWallet();
71                 BaseManager.Instance.ResetBase();
72                 HudManager.Instance.Reset();
73                 HudManager.Instance.Show();
74                 State = GameState.Prestart;
75                 EnemySpawnerManager.Instance.StopSpawning();
76                 EnemySpawnerManager.Instance.SceneEnabled(5); // Если
77             },
78             () => { HudManager.Instance.ShowPrestartButton(); }
79         );
80     }
81
82     public void StartPlaying()
83     {

```

```

84         if (State != GameState.Prestart) return;
85
86         State = GameState.Playing;
87         GameManager.Instance.StartGame();
88         SpeedGameManager.Instance.Resume();
89         AudioManager.Instance.PlayMusicType(MusicType.FightRandom);
90
91         EnemySpawnerManager.Instance.StartSpawning();
92     }
93
94     public void GameOver()
95     {
96         State = GameState.GameOver;
97
98         SpeedGameManager.Instance.Pause();
99         HudManager.Instance.ShowGameOverScreen();
100        GameManager.Instance.EndTimer();
101        EnemySpawnerManager.Instance.StopSpawning();
102        AudioManager.Instance.PlaySFXType(SoundDefaultEnum.GameOver);
103        TowerUIManager.Instance.HideInstant();
104    }
105
106    public void RestartGame()
107    {
108        if (restartLocked) return;
109        if (State != GameState.GameOver) return;
110
111        restartLocked = true;
112        GameManager.Instance.EndRun();
113        ArtefactManager.Instance.ResetInventory();
114        EnemySpawnerManager.Instance.StopSpawning();
115        LoadStage();
116    }
117
118    public void LeaveGame()
119    {
120        if (State == GameState.Loading) return;
121        //HudManager.Instance.HideDefeatScreen();
122        GameManager.Instance.EndRun();
123        ArtefactManager.Instance.ResetInventory();
124        EnemySpawnerManager.Instance.StopSpawning();
125        MenuGame();
126    }
127
128    public void MenuGame()
129    {
130        State = GameState.Loading;
131
132        SpeedGameManager.Instance.Pause();
133        EnemySpawnerManager.Instance.StopSpawning();
134
135        SceneLoader.Instance.LoadScene("MainMenu",
136            () =>
137            {
138                HudManager.Instance.Hide();
139                State = GameState.Menu;
140                AudioManager.Instance.PlayMusicType(MusicType.Menu);
141            }
142        );
143    }
144
145    public void EscMenu()
146    {
147        State = GameState.EscMenu;
148        SpeedGameManager.Instance.Pause();
149    }
150

```

```
151     public void ReturnPlaying()  
152     {  
153         State = GameState.Playing;  
154         SpeedGameManager.Instance.SetPastSpeed();  
155     }  
156 }
```

☰ Многие менеджеры получают доступ о текущем состоянии. Например монетки не будут падать, если текущее состояние не Playing. Соответственно, при выборе артефакта, на экране смерти, в меню - монетки не будут расти. Также дело обстоит с управлением временем через клавиатуру.

⚠ В скрипте на этапе написания документации найдены некоторые оплошности и ожидаются баги. Вскоре они будут исправлены!